

Pichulman Miguel Angel
TRABAJO PRACTICO 7

Introducción Teórica

En el desarrollo de aplicaciones con MongoDB, existen dos enfoques principales para gestionar la comunicación:

1. Driver Nativo: Provee la comunicación más directa con el motor de base de datos. Se destaca por ofrecer el máximo control sobre las consultas y una mayor eficiencia al no tener capas intermedias.
2. Mongoose (ODM): Actúa como una capa de abstracción sobre el driver. Permite definir esquemas para organizar los datos y forzar validaciones directamente en el código de la aplicación.

Parte 1: Implementación con Driver Nativo

El driver oficial (mongodb) es ideal para aplicaciones sencillas o procesos de alto volumen donde el rendimiento es crítico.

Instrucciones:

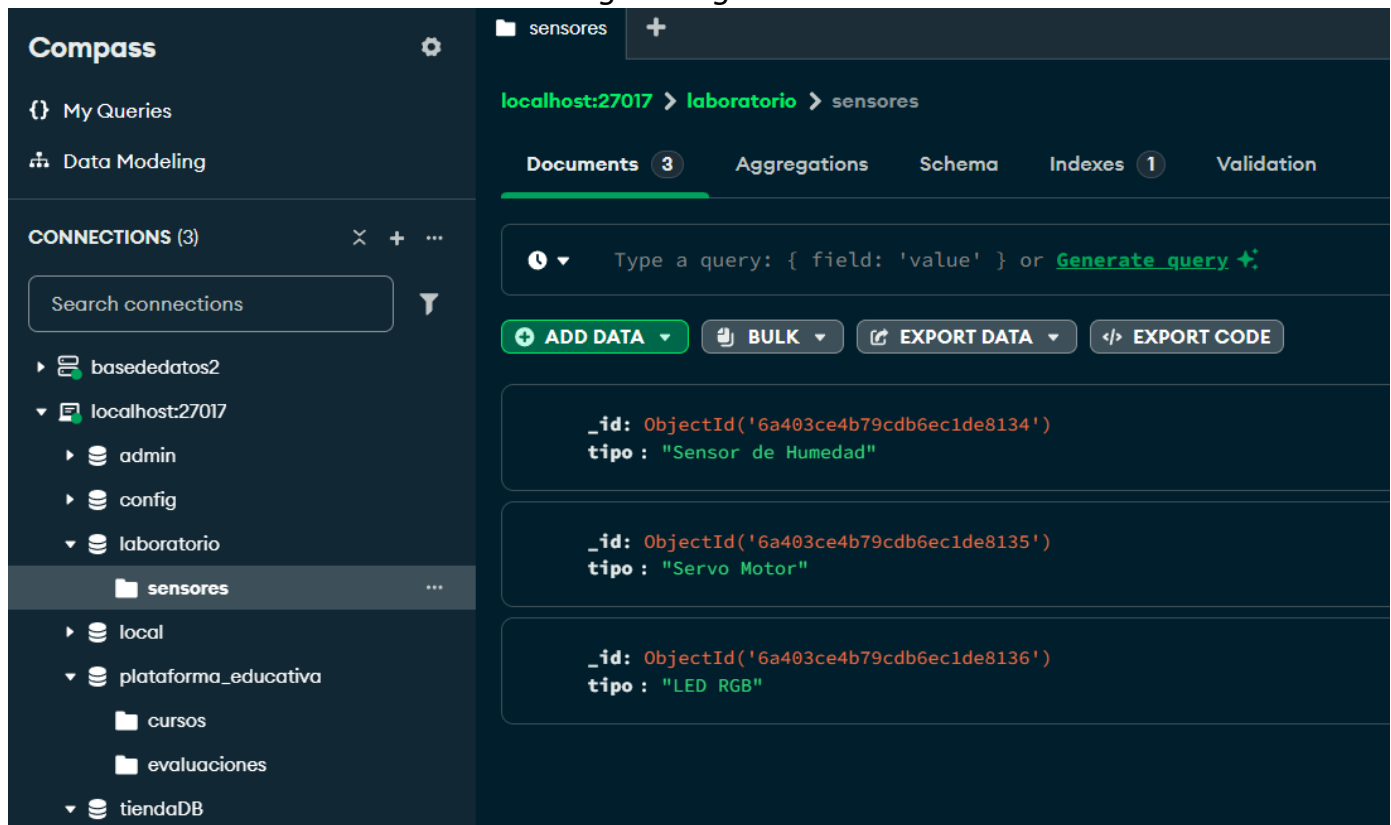
1. Inicialicen el proyecto e instalen el paquete: `npm install mongodb`.
2. Creen un archivo `driver_lab.js` para conectarse a la base de datos laboratorio.
3. Usando el método `insertMany`, carguen un array con tres sensores de robótica (ej: Sensor de Humedad, Servo Motor, LED RGB).

```
# driver_lab.js X
main
1 import { MongoClient } from 'mongodb';
2
3 const url = 'mongodb://localhost:27017/';
4 const client = new MongoClient(url);
5 const dbName = 'laboratorio'; // Conectando a la base de datos solicitada
6
7 async function main() {
8   await client.connect();
9   console.log('Conexión exitosa al servidor MongoDB (Driver Nativo)');
10  const db = client.db(dbName);
11  const collection = db.collection('sensores');
12
13  // Carga de array con tres sensores de robótica
14  const insertResult = await collection.insertMany([
15    { tipo: 'Sensor de Humedad' },
16    { tipo: 'Servo Motor' },
17    { tipo: 'LED RGB' }
18  ]);
19
20  console.log('Sensores insertados =>', insertResult);
21  return 'Parte 1: Completado.';
22 }
23
24 try {
25   const result = await main();
26   console.log(result);
27 } catch (error) {
28   console.error('Error en la conexión con MongoDB:', error);
29 } finally {
30   await client.close();
31 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\TercerCuatrimestre\BaseDeDatos2\7-conexion-BD-a-FrontEnd\TP7> node driver_lab.js
(node:3192) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module type of file:///C:/TercerCuatrimestre/BaseDeDatos2/7-conexion-BD-a-FrontEnd/TP7/driver_lab.js is not specified and it doesn't parse as CommonJS.
Reparsing as ES module because module syntax was detected. This incurs a performance overhead.
To eliminate this warning, add "type": "module" to C:\TercerCuatrimestre\BaseDeDatos2\7-conexion-BD-a-FrontEnd\TP7\package.json.
(Use 'node --trace-warnings ...' to show where the warning was created)
Conexión exitosa al servidor MongoDB (Driver Nativo)
Sensores insertados => {
  acknowledged: true,
  insertedCount: 3,
  insertedIds: {
    '0': new ObjectId('6a404472b603cde2903ee31'),
    '1': new ObjectId('6a404472b603cde2903ee32'),
    '2': new ObjectId('6a404472b603cde2903ee33')
  }
}
Parte 1: Completado.
PS C:\TercerCuatrimestre\BaseDeDatos2\7-conexion-BD-a-FrontEnd\TP7>
```

Pichulman Miguel Angel



Parte 2: Implementación con Mongoose (ODM)

Mongoose ayuda a mantener el orden y la coherencia en aplicaciones de mediana o gran escala.

Instrucciones:

1. Instalen el paquete: `npm install mongoose`.
2. Definan un Esquema (Schema) para los componentes que incluya:
 - o nombre: Tipo String y obligatorio (required: true).
 - o stock: Tipo Number.
3. Activen la opción `timestamps: true` para generar automáticamente los campos de fecha de creación y actualización.
4. Creen el modelo y realicen una inserción de prueba.



Pichulman Miguel Angel

```
JS driver_lab.js JS mongoose_lab.js X
JS mongoose_lab.js > ...
1  import mongoose from 'mongoose';
2
3  const url = 'mongodb://localhost:27017/';
4  const dbName = 'laboratorio';
5
6  // Definicion del Esquema
7  const componenteSchema = new mongoose.Schema({
8    nombre: {
9      type: String,
10     required: true
11   },
12   stock: {
13     type: Number
14   }
15 }, {
16   timestamps: true
17 });
18
19 // Creacion del modelo
20 const ComponenteModel = mongoose.model('Componente', componenteSchema);
21
22 async function main() {
23   // Conexión a MongoDB usando Mongoose
24   await mongoose.connect(`${url}${dbName}`);
25   console.log('Conexión exitosa a MongoDB con Mongoose');
26
27   // Insercion de prueba
28   const insertResult = await ComponenteModel.insertMany([
29     { nombre: 'Placa Arduino UNO', stock: 15 },
30     { nombre: 'Resistencia 10k', stock: 100 }
31   ]);
32
33   console.log('Componentes insertados =>', insertResult);
34   return 'Parte 2: Completado.';
35 }
36
37 try {
38   const result = await main();
39   console.log(result);
40 } catch (error) {
41   console.error('Error en la conexión o ejecución:', error);
42 } finally {
43   // Cierre de conexion
44   await mongoose.connection.close();
45 }
```

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA



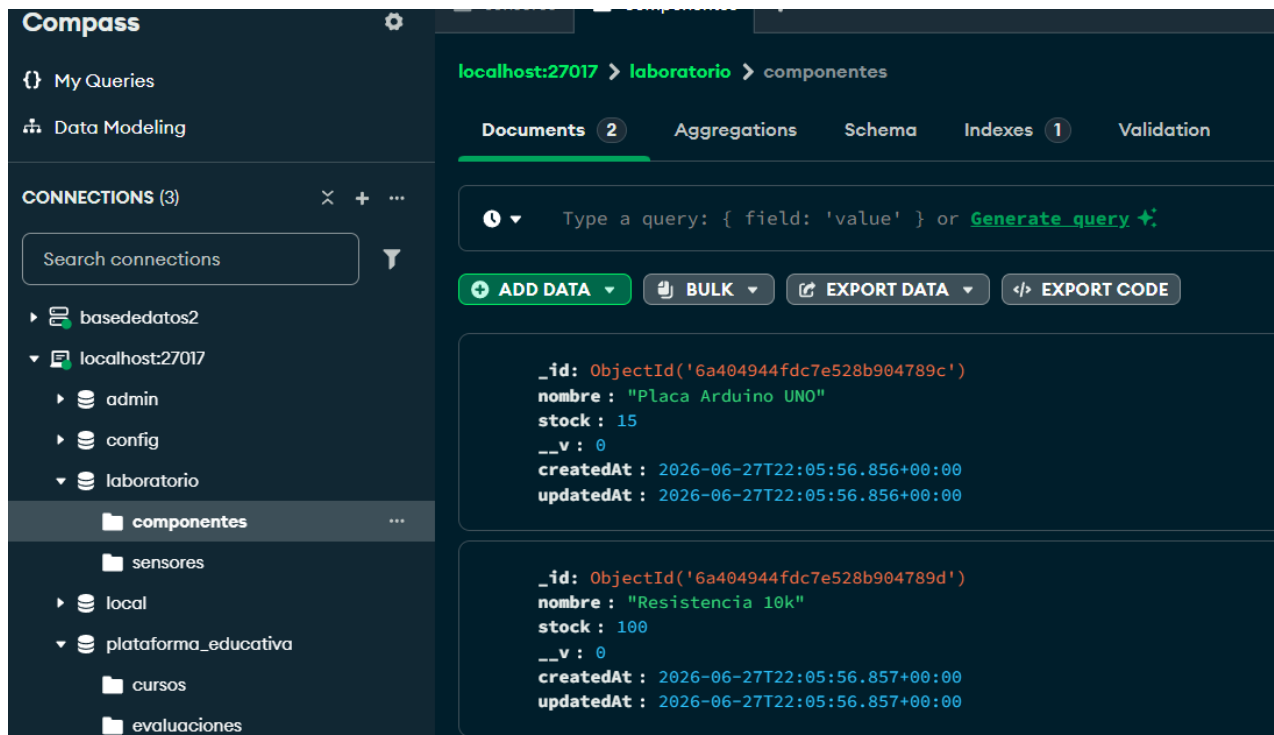
Pichulman Miguel Angel

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\TercerCuatrimestre\BaseDeDatos2\7-conexion-BD-a-FrontEnd\TP7> npm install mongoose

added 8 packages, and audited 21 packages in 6s

1 package is looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\TercerCuatrimestre\BaseDeDatos2\7-conexion-BD-a-FrontEnd\TP7> node mongoose_lab.js
(node:28184) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module type of file:///C:/TercerCuatrimestre/BaseDeDatos2/7-conexion-BD-a-FrontEnd/TP7/mongoose_lab.js is not specified and it doesn't parse as CommonJS.
Reparsing as ES module because module syntax was detected. This incurs a performance overhead.
To eliminate this warning, add "type": "module" to C:\TercerCuatrimestre\BaseDeDatos2\7-conexion-BD-a-FrontEnd\TP7\package.json.
(Use `node --trace-warnings ...` to show where the warning was created)
Conexión exitosa a MongoDB con Mongoose
Componentes insertados => [
  {
    nombre: 'Placa Arduino UNO',
    stock: 15,
    _id: new ObjectId('6a404944fdc7e528b904789c'),
    __v: 0,
    createdAt: 2026-06-27T22:05:56.856Z,
    updatedAt: 2026-06-27T22:05:56.856Z
  },
  {
    nombre: 'Resistencia 10k',
    stock: 100,
    _id: new ObjectId('6a404944fdc7e528b904789d'),
    __v: 0,
    createdAt: 2026-06-27T22:05:56.857Z,
    updatedAt: 2026-06-27T22:05:56.857Z
  }
]
Parte 2: Completado.
PS C:\TercerCuatrimestre\BaseDeDatos2\7-conexion-BD-a-FrontEnd\TP7> |
```



Pichulman Miguel Angel

Parte 3: Reflexión Final

Basándose en lo aprendido, respondan:

1. ¿Cuál es la función del driver como "traductor" entre la aplicación y MongoDB?.

El driver actúa como un puente de comunicación que toma las instrucciones escritas en el código de nuestra aplicación (en este caso, Node.js) y las traduce a los comandos directos que el motor de la base de datos puede entender y ejecutar. Es decir, provee la comunicación más directa posible sin capas intermedias.

2. ¿A qué formato convierte el driver las estructuras de datos nativas (como objetos de JavaScript) para enviarlas al servidor?.

El driver convierte los objetos nativos de JavaScript a un formato binario llamado **BSON** (Binary JSON). Este formato es el que MongoDB utiliza internamente para almacenar los documentos y transmitir los datos por la red de manera eficiente.

3. Según lo visto, ¿en qué situación de un proyecto de ingeniería elegirían usar el driver nativo en lugar de Mongoose?.

Se elegiría el driver nativo en situaciones donde el rendimiento es crítico y se manejan procesos de alto volumen de datos. Al no tener capas intermedias, el driver nativo ofrece la mayor eficiencia posible y permite tener el máximo control sobre las consultas complejas, a diferencia de Mongoose que añade una sobrecarga al validar esquemas.